॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

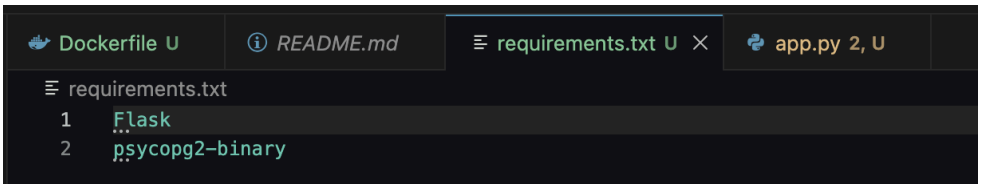
(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 10

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	**Yashmit Kotekar**
PRN No.	**23070122236**
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	23rd July, 2026
Title of Assignment:	Docker Volumes & Data Persistence
Theory:	Docker volumes provide persistent storage for containers by storing data outside the container's writable layer. Unlike container storage, Docker volumes remain intact even after a container is stopped or removed, allowing data to be shared across multiple container instances. By mounting a named volume to a directory inside the container, applications can read and write data that persists independently of the container lifecycle. This makes Docker volumes ideal for storing application data, logs, databases, and other important files that should not be lost when containers are recreated. **Commands Used** • Built the updated Docker image with a new version tag: docker build -t my-flask-app:v2 . • Created a Docker named volume: docker volume create my-app-data • Started a container with the named volume mounted: docker run -d -p 8082:5001 -v my-app-data:/app/data --name flask-volume-demo my-flask-app:v2 • Stopped the running container: docker stop flask-volume-demo • Removed the stopped container: docker rm flask-volume-demo • Started a new container using the same Docker volume: docker run -d -p 8082:5001 -v my-app-data:/app/data --name

	flask-volume-demo2 my-flask-app:v2
Implementation:	**Step 1: Modify the Python Application** Modified Flask Application — Updated the Flask application to write timestamped greetings to a log file inside the /app/data directory and configured the application to run on port 5001. **Step 2: Update the Dockerfile** Updated Dockerfile — Modified the Dockerfile to expose port 5001, matching the updated Flask application configuration. **Step 3: Build the New Docker Image** Built Updated Docker Image — Built a new Docker image named my-flask-app:v2 containing the modified Flask application configured for Docker volume-based data persistence. **Step 4: Create a Docker Named Volume** Created Docker Volume — Created a Docker named volume called my-app-data to store application data independently of the container, ensuring data persistence across container restarts and removal. **Step 5: Access the Application and Write Data** Verified Data Logging — Accessed the Flask application through http://localhost:8082 and refreshed the page multiple times to verify that timestamped greetings were successfully written to the persistent log file stored in the Docker volume. **Step 6: Stop and Remove the Container** Stopped and Removed Container — Stopped the flask-volume-demo container and removed it using the docker stop and docker rm commands while preserving the application data stored in the Docker named volume. **Step 7: Verify Data Persistence** Verified Data Persistence — Accessed the Flask application after starting a new container with the same Docker named volume and confirmed that the previously stored log entries were retained. This demonstrated successful data persistence across multiple container instances. 

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python","app.py"]
```

Dockerfile U | README.md | requirements.txt U | app.py 2, U

app.py > ...

```
from flask import Flask
import psycopg2
import time

app = Flask(__name__)

while True:
    try:
        conn = psycopg2.connect(
            host="db",
            database="mydatabase",
            user="postgres",
            password="postgres"
        )
        break
    except Exception:
        print("Waiting for PostgreSQL...")
        time.sleep(2)

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS messages(
    id SERIAL PRIMARY KEY,
    message TEXT
)
""")

conn.commit()

@app.route("/")
```

```
def hello():

    cursor.execute(
        "INSERT INTO messages(message) VALUES(%s)",
        ("Hello from Flask!",)
    )

    conn.commit()

    cursor.execute("SELECT COUNT(*) FROM messages")

    count = cursor.fetchone()[0]

    return f"""
    <h1>Hello World!</h1>
    <h2>Database Connected Successfully</h2>
    <h3>Total Records: {count}</h3>
    """

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Subject Faculty Name: Prof. Arijit Dutta